

AULA 8

Ações a fundo: o ciclo de vida de um objetivo

Etapa 4 · terça, 08/09/2026 · sala SJ205 · Prof. Dácio Moreira de Souza

Onde paramos

- Os cartões funcionaram: folha levantada, serviço chamado, banner no terminal
- Serviço é pergunta e resposta — em milissegundos. Por isso funcionou
- Troque o comando: "vá até a bancada e me avise quando chegar"
- O serviço trava quem pediu por 30 s, em silêncio, e não pode ser interrompido
- Foi aí que o nome ACTION apareceu. É daí que a aula de hoje começa



**Um objetivo não é uma mensagem.
É uma entidade —
com identidade e com estado.**

Todo o resto decorre disso.

PARTE 1

O que é, de fato, uma action

A comparação que ajuda a decidir

Tópico e serviço

- a coisa trocada NÃO tem identidade
- mensagem é evento anônimo
- a chamada de serviço termina e some
- não há estado observável
- ninguém pode se referir a ela depois

Action

- cada objetivo tem um goal_id
- tem máquina de estados observável
- servidor OU cliente podem terminar
- existe resultado parcial: feedback
- dá para falar dele depois de criado

Depois de mandar, eu vou querer me referir a isso de novo?

Se sim — acompanhar, cancelar, saber como terminou — é action. Se não, tópico ou serviço, com muito menos código.

A máquina de estados — e quem provoca cada seta

- → Rejeitado · o SERVIDOR, antes de gastar nada · `GoalResponse.REJECT`
- → Aceito → Executando · o SERVIDOR · entra o `execute_callback`
- Executando → Bem-sucedido · o SERVIDOR · `succeed()`
- Executando → Abortado · o SERVIDOR, porque desistiu · `abort()`
- Executando → Cancelando · o CLIENTE · `cancel_goal_async()`
- Cancelando → Cancelado · o SERVIDOR, obedecendo · `canceled()`

Duas leituras da máquina

- REJEITADO não é fim de execução — é a AUSÊNCIA dela. Nada foi gasto
- Servidor que aceita tudo e depois falha desperdiça trabalho e informa pior
- CANCELANDO é um estado, não um instante
- Entre o pedido e o Cancelado existe tempo — e num robô é onde mora a segurança

PARTE 2

Os três verbos de término

Mesmo fim, motivos opostos

`canceled()`

- "mudei de ideia"
- o sistema está saudável
- quem pediu sabe por quê
- decisão do CLIENTE, honrada pelo servidor

`abort()`

- "não deu — e não foi você que pediu"
- algo quebrou
- quem pediu precisa decidir o que fazer
- decisão do SERVIDOR, contra a vontade do cliente

Um cliente que trata os dois igual toma a decisão errada em um dos casos.

Ou insiste numa tarefa que o operador cancelou de propósito, ou desiste em silêncio de uma falha que dava para corrigir.

Abortar ao vivo: mate a percepção

```
# terminal 1
ros2 launch aula07_acoes varredura.launch.py

# terminal 2 -- varredura longa
ros2 action send_goal /varrer_cena \
  pb_interfaces/action/VarrerCena \
  "{duracao_s: 60.0, classe_alvo: 'objeto_alvo',
    minimo_para_parar: 0}" --feedback

# terminal 3 -- mate a percepcao no meio
pkill -f 'aula06_percepcao.*detector'
```

Em 3 s o objetivo termina com ABORTED. Varrer a cena sem imagem não é uma varredura ruim: é uma varredura que não está acontecendo.

Um objetivo por vez — política, não limitação

- Por padrão o rclpy aceita quantos objetivos chegarem e executa em paralelo
- Num robô físico isso quase sempre está errado: ele tem um braço só
- O `goal_callback` do exemplo rejeita enquanto houver varredura em curso
- E o comentário admite onde a solução simples quebra: dois pedidos no mesmo ms passam
- Saber onde a sua solução simples quebra faz parte de escolher a solução simples

PARTE 3

A ordem dentro do laço

A ordem importa mais que o conteúdo

```
while True:
    if goal_handle.is_cancel_requested:      # 1. cancelamento SEMPRE primeiro
        goal_handle.canceled()
        return resultado_parcial
    if percepcao_muda:                       # 2. desistir e decisao do servidor
        goal_handle.abort()
        return resultado_parcial
    ...                                     # 3. o trabalho
    if condicao_de_sucesso:                  # 4. sucesso antecipado
        goal_handle.succeed()
        return resultado
    goal_handle.publish_feedback(fb)         # 5. feedback
    time.sleep(PERIODO_S)                   # 6. dormir um pouco
```

As duas linhas que enganam

```
# num terminal SEM source install/setup.bash:

ros2 action list                # /varrer_cena                <- FUNCIONA
ros2 action info /varrer_cena -t # tipo certo                <- FUNCIONA

ros2 action send_goal ...       # The passed action type is invalid
ros2 run aula07_acoes cliente   # Package not found
```

As duas primeiras perguntam ao GRAFO, pela rede. As duas últimas precisam do seu código na máquina. A ferramenta que funciona te convence de que o ambiente está certo.

Diagnóstico em cinco segundos

```
python3 -c "from pb_interfaces.action import VarrerCena; print('ok')"  
ros2 pkg list | grep aula07_acoes
```

```
# se qualquer um falhar, a cura é sempre a mesma, NESTE terminal:  
cd ~/projeto-pb-SEU-USUARIO/ros2_ws  
source install/setup.bash
```

A mensagem "action type is invalid" manda olhar para o .action, que está perfeito.

PARTE 4

O bug que não dá erro



**Tudo certo no código da action.
Pede cancelamento.
Não cancela.
E não aparece erro nenhum.**

A causa está fora do código da action

- Por padrão o rclpy roda com um executor de UMA thread só
- O `execute_callback` ocupa essa thread até terminar
- O pedido de cancelamento chega, entra na fila, e espera a thread ficar livre
- A thread só fica livre quando a tarefa acaba sozinha
- `is_cancel_requested` nunca vira verdadeiro, porque nada nunca o setou

A cura são duas linhas — e as duas são necessárias

```
from rclpy.callback_groups import ReentrantCallbackGroup
from rclpy.executors import MultiThreadedExecutor

callback_group=ReentrantCallbackGroup()           # no ActionServer
rclpy.spin(no, executor=MultiThreadedExecutor())  # no main()
```

Comente as duas, recompile, e tente cancelar. Ver a coisa NÃO funcionar, sem nenhuma mensagem, é o que fixa a lição.

O comportamento ausente não produz erro.

Teste procura erro. Funcionalidade que simplesmente não acontece só aparece se você a exercitar de propósito. Por isso o G2.3 pede DEMONSTRAÇÃO.

PARTE 5

Introdução à percepção veicular

Esse objeto que a câmera viu está a que distância da roda?

O Demo 1 respondia em PIXELS. Um robô não anda em pixels.

Três coisas que a percepção veicular junta

- DETECTAR melhor que por cor — carro não é vermelho; placa de pare e pano vermelho são
- → modelo treinado traz uma obrigação nova: DECLARAR uma métrica (G2.4, 19/09)
- SITUAR a detecção no espaço do robô — isso é TF; onde o sensor está montado é URDF
- → sem isso, "vi um obstáculo" não vira "freie": ninguém sabe se está a 2 m ou 20 cm
- DECIDIR com atraso — entre ver e agir existe tempo. A 30 km/h, atraso é distância

Leitura prévia para 15/09 — comece pela instalação

```
sudo apt install -y \  
  ros-humble-joint-state-publisher-gui \  
  ros-humble-xacro ros-humble-tf2-tools liburdfdom-tools  
  
# depois, o tutorial completo no site:  
# tutoriais/urdf-tf-rviz2/  
  
check_urdf meu_robo.urdf          # valide ANTES de rodar  
ros2 run tf2_tools view_frames     # a arvore, em PDF
```

A Aula 9 será clínica, não primeira exposição. Quem chegar sem o tutorial vai gastar a aula instalando pacote.

Antes da próxima terça

- G2.2 vence 12/09 · G2.3 vence 16/09 — os dois saem desta aula
- A tarefa ganhou dois itens: REJEITAR objetivo inválido e ABORTAR quando ficar impossível
- Grave os TRÊS desfechos: sucesso, cancelamento e abort
- NavigateToPose do Nav2 é uma action: feedback = distância restante, abort = sem caminho
- Aula 9, 15/09: detecção treinada com métrica (G2.4) + clínica de URDF e TF (G2.5)
- FAÇA o tutorial de URDF/TF antes de 15/09 — ele começa na instalação